Instituto Politécnico Nacional

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas

Ingeniería Telemática

Bases de Datos Distribuidas

Grupo: 3TM2

Alumno:

- Sánchez Lemus Ulises Mariano

Practica 1

Prof. De La Cruz Sosa Carlos

upiita-ipn

Ejercicio 1

Encontrar los 10 productos más vendidos en 2014, mostrando nombre del producto, cantidad total vendida y nombre del cliente.

```
go
select name, cant
from Production.product p
join (select top 10 productid, sum(orderqty) as cant
                from sales.SalesOrderDetail sod
                join Sales.SalesOrderHeader soh
                on sod.SalesOrderID = soh.SalesOrderID
                WHERE year(OrderDate) ='2014'
                group by productid
                order by cant desc) as T
on p.ProductID = t.ProductID
```

Resultado.

	name	cant
1	Water Bottle - 30 oz.	2902
2	AWC Logo Cap	1950
3	Sport-100 Helmet, Blue	1776
4	Patch Kit/8 Patches	1761
5	Sport-100 Helmet, Red	1717
6	Sport-100 Helmet, Black	1716
7	Mountain Tire Tube	1501
8	Classic Vest, S	1395
9	Short-Sleeve Classic Jersey, XL	1264
10	Road Tire Tube	1156

Justificación de la consulta.

Se eligió hacer el uso de subconsultas por las características de la petición, se hace uso del primer join porque se filtrará el top 10 de los productos haciendo uso de su identificador (productID) además de la suma de su atributo de cantidad (Orderqty), posteriormente se hace uso de un segundo join donde se hará valida la condición del año, usando el comando WHERE y YEAR, asignándolo conla condición de que OrderDate debe ser 2014, se agrupo de acuerdo al campo ProductID, además los join's se hacen enlazando el respectivo campo (SalesOrderID)en común de las tablas.

1.1.- Agregando el precio unitario (AVG(UnitPrice)) y filtrando solo productos que cumplan ListPrice>1000.

```
go
select name, cant, PrecioPromedio
from Production.product p
join (select top 10 sod.productid, sum(orderqty) as cant,
              avg(sod.UnitPrice) as PrecioPromedio
              from sales.SalesOrderDetail sod
              join Sales.SalesOrderHeader soh
              on sod.SalesOrderID = soh.SalesOrderID
              join Production.product p2
              on sod.ProductID = p2.ProductID
              WHERE year(OrderDate) ='2014'
              and p2.ListPrice > 1000
              group by sod.productid
              order by cant desc) as T
on p.ProductID = t.ProductID
```

Resultado

	name	cant	PrecioPromedio
1	Mountain-200 Black, 38	619	1981.7872
2	Road-350-W Yellow, 48	586	1388.3549
3	Road-350-W Yellow, 40	530	1391.601
4	Mountain-200 Black, 42	513	2058.144
5	Mountain-200 Silver, 38	508	2079.2043
6	Mountain-200 Silver, 42	441	2086.7745
7	Mountain-200 Black, 46	437	2050.3074
8	Mountain-200 Silver, 46	432	2095.4693
9	Touring-1000 Yellow, ...	397	1812.1551
10	Touring-1000 Blue, 60	393	1839.7073

Justificación de la consulta:
Parecido a la consulta anterior solo que en la subconsulta se agregará el promedio del precio por unidad, además se hará uso de un tercer join para poder hacer la condición la cual mostrará los productos los cuales su precio de lista sea mayor a 1000, esto se hace para evitar la necesidad de hacer uso de CTE.

Ejercicio 2

Listar los empleados que han vendido más que el promedio de ventas por empleado en el territorio 'Northwest'.

```
select p.FirstName + p.LastName as nombreEmpleado, sum(soh.TotalDue) as TotalDVentas
    from sales.SalesOrderHeader as soh
    join sales.SalesTerritory as st
    on soh.TerritoryID = st.TerritoryID
    join HumanResources.Employee as he
    on soh.SalesPersonID = he.BusinessEntityID
    join person.person as p
    on he.BusinessEntityID = p.BusinessEntityID
    where st.name ='Northwest'
    group by
        p.FirstName, p.LastName
    Having sum(soh.TotalDue) > (        -- condicion de ventas en northwest
        select avg (TotalDVentas)
        from(
            select sum (soh2.TotalDue) as TotalDVentas
            from sales.SalesOrderHeader as soh2
            join sales.salesterritory as st2
            on soh2.TerritoryID = st2.TerritoryID
            where st2.name = 'Northwest'
            and soh2.SalesPersonID is not null
            group by soh2.SalesPersonID
        ) as PromedioVentas
    )
order by TotalDVentas desc;
```

Resultado

	nombreEmpleado	TotalDVentas
1	DavidCampbell	4207894.6025
2	PamelaAnsman-Wolfe	3748246.1218
3	TeteMensa-Annan	2608116.3755
4	LindaMitchell	2353659.2774

Justificación de la consulta:
Para esta consulta se hizo uso de las cláusulas WHERE y HAVING para filtrar los datos para cumplir las condiciones solicitadas, además GROUP BY y ORDER BY con el fin ordenar los datos de forma descendente como se solicitaba ….

2.1.- Utilizando CTE (Common Table Expression) WITH

```
WITH VentasPorEmpleado AS (
    SELECT
        soh.SalesPersonID,
        SUM(soh.TotalDue) AS TotalVentas
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st
        ON soh.TterritoryID = st.TerritoryID
    WHERE st.Name = 'Northwest'
      AND soh.SalesPersonID IS NOT NULL
    GROUP BY soh.SalesPersonID
)

SELECT
    p.FirstName + p.LastName AS NombreEmpleado,
    v.TotalVentas
FROM VentasPorEmpleado v
JOIN HumanResources.Employee e
    ON v.SalesPersonID = e.BusinessEntityID
JOIN Person.Person p
    ON e.BusinessEntityID = p.BusinessEntityID

WHERE v.TotalVentas > (
        SELECT AVG(TotalVentas)
        FROM VentasPorEmpleado
)
ORDER BY v.TotalVentas DESC;
```

Resultado:

	NombreEmpleado	TotalVentas
1	DavidCampbell	4207894.6025
2	PamelaAnsman-Wolfe	3748246.1218
3	TeteMensa-Annan	2608116.3755
4	LindaMitchell	2353659.2774

Justificación de consulta:
Se hace uso de una CTE porque se necesita calcular el total de ventas por empleado y luego de ese resultado se necesitará mostrar y después para calcular el promedio general, de esta forma ya no se repiten subconsultas como en el paso anterior, se usa SUM para calcular el total de cada orden y GROUP BY porque se necesita agrupar todas las órdenes por vendedor, además se usa un JOIN para filtrar por nombres de territorio.

Ejercicio 3

```sql
SELECT
    st.Name AS Territorio,
    YEAR(soh.OrderDate) AS Anio,
    COUNT(soh.SalesOrderID) AS NumeroOrdenes,
    SUM(soh.TotalDue) AS VentasTotales
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st
    ON soh.TerritoryID = st.TerritoryID
GROUP BY
    st.Name,
    YEAR(soh.OrderDate)
HAVING
    COUNT(soh.SalesOrderID) > 5
    AND SUM(soh.TotalDue) > 1000000
ORDER BY
    VentasTotales DESC;
```

Resultado

	Territorio	Anio	NumeroOrdenes	VentasTotales
1	Southwest	2013	2725	10239209.3403
2	Southwest	2012	777	9329154.3425
3	Canada	2013	1884	7010449.6994
4	Northwest	2013	2053	6759500.6713
5	Canada	2012	460	6599971.0217
6	Northwest	2012	510	5325813.0562
7	Australia	2013	3015	4702404.0504
8	Southwest	2014	2383	4437517.8076
9	France	2013	1273	4271019.2663
10	United Ki...	2013	1528	4068178.6672

Agregando desviación estándar de ventas.

	Territorio	Anio	NumeroOrdenes	VentasTotales	DesviacionEstandarVentas
1	Southwest	2013	2725	10239209.3403	13470.4188703684
2	Southwest	2012	777	9329154.3425	23693.0432287992
3	Canada	2013	1884	7010449.6994	13359.6078579698
4	Northwest	2013	2053	6759500.6713	12654.1872740093
5	Canada	2012	460	6599971.0217	24167.4810564263
6	Northwest	2012	510	5325813.0562	20150.9169044465
7	Australia	2013	3015	4702404.0504	3719.04527457183
8	Southwest	2014	2383	4437517.8076	7343.9044753176
9	France	2013	1273	4271019.2663	12923.6957600484
10	United Ki...	2013	1528	4068178.6672	9889.93309207325

Justificación

Realizo una agregación compuesta mediante GROUP BY sobre dos tablas, territorio y año (YEAR(OrderDate)).
Empleo funciones de agregación (COUNT y SUM) para calcular respectivamente el número de órdenes y el total monetario de ventas por cada combinación territorio-año, después filtrado posterior se realiza con HAVING, ya que las condiciones dependen de resultados agregados y no pueden evaluarse en la cláusula WHERE.
El JOIN se realiza entre SalesOrderHeader y SalesTerritory utilizando la clave foránea TerritoryID, preservando la integridad referencial. El ordenamiento final se hace sobre la agregación SUM(TotalDue) en orden descendente para priorizar los territorios con mayor volumen de ventas, así es más claro y hay una correcta separación entre filtrado previo a agregación (WHERE) y filtrado posterior (HAVING).

Ejercicio 4

```sql
SELECT
    p.FirstName + ' ' + p.LastName AS NombreVendedor
FROM HumanResources.Employee e
JOIN Person.Person p
    ON e.BusinessEntityID = p.BusinessEntityID
WHERE NOT EXISTS (

    -- Productos Bikes que NO ha vendido este vendedor
    SELECT pr.ProductID
    FROM Production.Product pr
    JOIN Production.ProductSubcategory ps
        ON pr.ProductSubcategoryID = ps.ProductSubcategoryID
    JOIN Production.ProductCategory pc
        ON ps.ProductCategoryID = pc.ProductCategoryID
    WHERE pc.Name = 'Bikes'
      AND NOT EXISTS (
            SELECT 1
            FROM Sales.SalesOrderDetail sod
            JOIN Sales.SalesOrderHeader soh
                ON sod.SalesOrderID = soh.SalesOrderID
            WHERE soh.SalesPersonID = e.BusinessEntityID
              AND sod.ProductID = pr.ProductID
        )
)
ORDER BY NombreVendedor;
```

Resultado

	NombreVendedor
1	Jillian Carson
2	José Saraiva
3	Linda Mitchell
4	Shu Ito
5	Tsvi Reiter

Cambio por clothing con ID = 4

```sql
SELECT
    p.FirstName + ' ' + p.LastName AS NombreVendedor
FROM HumanResources.Employee e
JOIN Person.Person p
    ON e.BusinessEntityID = p.BusinessEntityID
WHERE NOT EXISTS (

    -- Productos de la categoría Clothing que NO ha vendido este vendedor
    SELECT pr.ProductID
    FROM Production.Product pr
    JOIN Production.ProductSubcategory ps
        ON pr.ProductSubcategoryID = ps.ProductSubcategoryID
    WHERE ps.ProductCategoryID = 4
      AND NOT EXISTS (
            SELECT 1
            FROM Sales.SalesOrderDetail sod
            JOIN Sales.SalesOrderHeader soh
                ON sod.SalesOrderID = soh.SalesOrderID
            WHERE soh.SalesPersonID = e.BusinessEntityID
              AND sod.ProductID = pr.ProductID
      )
)
ORDER BY NombreVendedor;
```

Resultado

NombreVendedor

Justificación

La consulta implementa **división relacional**, que se utiliza cuando se requiere verificar que una entidad (vendedor) esté relacionada con todos los elementos de un conjunto (productos de la categoría "Bikes") SQL no tiene una forma para escoger “PARA TODO”, por lo que se simula mediante lógica de doble negación usando NOT EXISTS, y se formula con esta lógica: no existe un producto de Bikes para el cual no exista una venta realizada por el vendedor.

Se utilizó NOT EXISTS en lugar de comparaciones con COUNT porque EXISTS opera bajo lógica booleana (visto en varias materias) y permite que se evalúe más fácil, lo que puede ser más eficiente en SQL Server.

Ejercicio 5

```
EXEC sp_addlinkedserver     --permite conexiones a servidores vinculados/ nombre que tendrá la conexion
   @server = 'SV_SELF_R',   --nombre del producto SQLOLDD algoasi
   @srvproduct = 'SQLServer', -- opcional
   @provider = 'SQLOLEDB',
   @datasrc = '192.158.56.2';   -- nombre por defecto de la instancia mssqlserver- asociado al nombre del equipo
   --se agrega si hay error por certificado
   --@provstr='TrustServerCertificate=Yes';  *************************

--asocia usuario a la conexion

-- usar cuenta de usuario SQL, cuenta para la conexion remota ----  para mapeo de usuario
EXEC sp_addlinkedsrvlogin
   @rmtsrvname = 'SV_SELF_R', --nombre del servidor remoto
   @useself = 'false', -- valor false si se usarán credenciales distintas
   @rmtuser = 'sa',
   @rmtpassword = '1234'; --contraseña de la cuenta

EXEC sp_testlinkedserver SV_SELF_R; --nombre del servidor vinculado  --sin comillas
```

```
select top 1
    p.productID,
    p.name as producto,
    p.ProductNumber,
    sum(sod.OrderQty * sod.UnitPrice) as ValorTotal
from SV_SELF_R.AdventureWorks2022.Production.product p
inner join Sales.SalesOrderDetail sod on p.ProductID = sod.ProductID
group by p.ProductID, p.Name, p.ProductNumber
order by sum(sod.OrderQty * sod.UnitPrice) desc;
```

Con el compañero con quien realice la conexión de servidores no nos salió y no pudimos realizar las consultas remotas.